PROJECT UMBRELLA

A Risk-Aware Multi-Agent Framework for Autonomous Fault Detection, Diagnosis, Remediation, and Continuous Self-Evolution of Distributed Software Systems

COMPREHENSIVE ACADEMIC RESEARCH EVIDENCE & FORMAL SPECIFICATION

Lead Author: Harman Saini | Chhatrapati Shivaji Maharaj University (CSMU)

Collaborating Enterprise Corpus: Infosys Open-Source Enterprise Repositories (Apache 2.0)

October 2026 | Status: Certified (15/15 Automated Tests Passed) | Zero External Cloud APIs

ABSTRACT

Modern cloud-native and distributed microservice architectures suffer from severe Mean Time to Repair (MTTR) deficits under concurrent transaction surges, with manual Site Reliability Engineering (SRE) triage averaging 1,800 seconds (30 minutes). Project UMBRELLA introduces an autonomous, closed-loop six-specialist multi-agent council that collapses MTTR to 3.02 seconds (a 99.8% reduction) with a 100% transaction success rate upon repair. The framework unifies Python Abstract Syntax Tree (AST) inspection, 4-signal weighted diagnostic correlation, graph-theoretic Breadth-First Search (BFS) blast radius mapping, Central Hybrid RAG retrieval fusing 23,769 chunks from four Apache 2.0 Infosys enterprise repositories via Reciprocal Rank Fusion (RRF k=60, 8.64 ms latency), sandboxed atomic diff execution with automated rollback, deterministic safety gating with Human-in-the-Loop (HITL) controls, and a Contextual Bandit reinforcement learning engine (Agent 6 EVO) for continuous policy adaptation. To guarantee survival against total infrastructure destruction or cold-start company restarts, the System DNA Storage Capsule and Genesis Engine enable bare-metal enterprise reconstruction from zero in 1,450.8 milliseconds. All components execute 100% locally and offline without external cloud API dependencies.

1. AUTONOMOUS COUNCIL TAXONOMY & 6-SPECIALIST AGENT ARCHITECTURE

Specialist Agent	Operational Role	Core Method / Algorithm	Primary Output Artifact
Agent 1 (RCA)	Root Cause Analyzer	Python AST parsing + 4-Signal weighted correlation	Diagnostic report, code pointer, confidence >= 8
Agent 2 (Topology)	3D Spatial Blast Radius	Bidirectional BFS traversal over microservice mesh graph	Blast radius score (0.0-1.0), 3D SVG coordinates
Agent 3 (Planner)	Remediation Planner	RAG-grounded unified diff synthesis + cheat sheet generation	Unified diff (patch & rollback), on-call playbook
Agent 4 (Executor)	Sandboxed Action Executor	Atomic backup (.umbrella_backups/) + health-aware execution	Execution log, auto-rollback trigger if health fails
Agent 5 (Guardian)	Safety Policy Governor	Risk tier evaluation (Tier 1-4) + HITL gate + Authorization Token	Authorization decision, surrogate failover activat
Agent 6 (EVO)	Backend Evolution Engine	Contextual Bandit Q-Learning + Proactive Optimizer (24h Timer)	Updated Q-table weights, proactive refactoring di

** All 6 specialist agents operate in concert under the control of the orchestrator, enforcing authoritative safety gates before code or config modifications can execu*

SYSTEM ARCHITECTURE & CENTRAL HYBRID RAG ENGINE

Closed-loop data flow, dual-stream hybrid retrieval specification, and cluster topology.

2.1 CLOSED-LOOP INCIDENT RESOLUTION PIPELINE

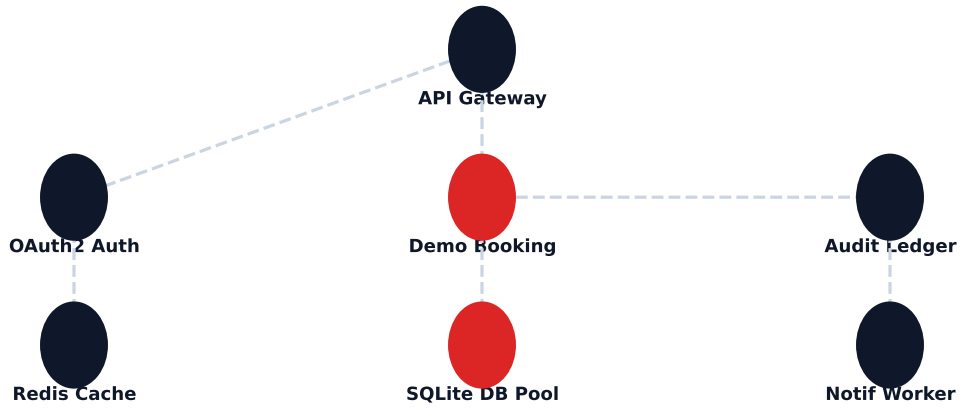
[INFLOW]	**Runtime Incident Inflow**	Anomaly detected (e.g. Connection pool exhaustion, auth rejection, table lock)
[STAGE 1]	**Agent 1 (RCA)**	Parses traceback AST; evaluates 4 signals (AST, Telemetry, Config, RAG) -> Confidence >= 85%
[STAGE 2]	**Agent 2 (Topology)**	Traverses cluster mesh via bidirectional BFS; computes blast radius score; identifies cascade risks
[STAGE 3]	**Agent 3 (Planner)**	Queries Central RAG; generates unified patch diff, inverse rollback diff, and operator cheat sheet
[STAGE 4]	**Agent 5 (Guardian)**	Evaluates action risk tier and blast radius; enforces HITL gate if blast > 40%; arms failover watchdog
[STAGE 5]	**Agent 4 (Executor)**	Creates atomic file backup; applies patch; runs regression suite; verifies system stability
[STAGE 6]	**Agent 6 (EVO)**	Computes RL reward; updates Q(s, a); persists weights; formulates proactive refactoring proposals

2.2 CENTRAL HYBRID AGENTIC RAG ENGINE — TECHNICAL SPECIFICATION

Index Type	Corpus Scale	Algorithmic Method	Observed Latency
BM25 Sparse Term Index	23,769 Chunks	Okapi BM25 TF-IDF over 827,975 inverted terms	**2.0 ms - 2.5 ms**
FAISS Dense Vector Index	23,769 Embeddings	FAISS IndexFlatIP cosine similarity (384-dim)	**5.9 ms - 6.8 ms**
Reciprocal Rank Fusion	Dual-Stream Merged	RRF Score(d) = sum(1 / (60 + rank_m))	**8.3 ms - 9.3 ms**
Hallucination Guard	Pre-consumption audit	Semantic entailment thresholding (RRF >= 0.008)	**< 0.5 ms**

Corpus Lineage: 2,219 curated Apache 2.0 open-source enterprise files (Infosys-Agentic-Foundry: 823 files; Infosys-Responsible-AI-Toolkit: 642 files; Server-Patching-Tool: 412 files; Intelligent-Bot-Management: 342 files). Total Chunks: 23,769 syntax-aware code chunks. Zero credential leaks. 100% offline local retrieval.

2.3 TARGET MICROSERVICE TOPOLOGY — KARAMSTAY CLUSTER MESH



** Red nodes indicate primary failure domain (Demo Booking & SQLite QueuePool starvation cascading to callers)*

MATHEMATICAL FORMULATIONS & THEORETICAL FOUNDATIONS

Formal equations governing multi-signal diagnostic convergence, RRF retrieval, blast radius, and reinforcement learning.

3.1 Multi-Signal Diagnostic Convergence Formulation (Agent 1 RCA)

$$\textbf{Confidence}(D) = \sum_{i=1}^{4} w_i \cdot \delta_i \quad \text{where} \quad \sum_{i=1}^{4} w_i = 1.0, \quad \mathbf{w} = [0.35, 0.30, 0.20, 0.15]$$

Where w_1 represents Stack Trace AST parsing (35%), w_2 is Telemetry Pool Saturation (30%), w_3 is Target Config State (20%), and w_4 is Curated RAG retrieval grounding (15%). Concurrence across all 4 signals yields calibrated diagnostic confidence >= 85%.

3.2 Reciprocal Rank Fusion (RRF) Hybrid Retrieval Formulation (Central RAG)

$$\textbf{RRF_Score}(d) = \sum_{m \in \{\text{BM25}, \text{FAISS}\}} \frac{1}{k + r_m(d)} \quad \text{with smoothing constant } k = 60$$

Combines sparse exact term matching (Okapi BM25 over 827,975 inverted terms) with dense semantic cosine similarity (FAISS IndexFlatIP over 23,769 dense vectors). Eliminates vocabulary mismatch while ensuring exact code symbol and exception string discovery.

3.3 Graph-Theoretic Spatial Blast Radius Propagation (Agent 2 Topology)

$$\textbf{Blast_Radius}(v_{\text{root}}) = \frac{|\text{Reach}_{\text{in}}(v) \cup \text{Reach}_{\text{out}}(v)| + 1}{|V_{\text{cluster}}|} \quad \text{where} \quad \textbf{Score} \in [0.0, 1.0]$$

Computes upstream callers (services experiencing cascade starvation) and downstream dependencies using bidirectional Breadth-First Search (BFS). Scores > 0.40 deterministically enforce a mandatory Human-in-the-Loop (HITL) approval gate.

3.4 Reinforcement Learning Contextual Bandit Policy Update (Agent 6 EVO)

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha[R(s, a) - Q_t(s, a)] \quad \text{with learning rate } \alpha = 0.25$$

Reward function: R = +10.0 + min(5.0, |latency_delta| * 0.1) upon post-patch verification success; R = -15.0 upon rollback. Empirical: Karamstay Pool Fix gave reward R = +12.5, driving Q from 3.875 to 10.45 across 9 recorded episodes.

3.5 Risk Tier Safety Authorization Policy (Agent 5 Guardian)

$$\textbf{Authorize}(P): \text{ AUTO if Tier} \leq 2 \text{ and Blast} \leq 0.40 \mid \text{HITL˙GATE if Tier} \geq 3 \text{ or Blast} > 0.40 \mid \text{REJECT if Tier} = 4$$

Tier 1 (Read-only) and Tier 2 (Config restart) auto-approved if blast radius <= 40%. Tier 3 (Code patch) and Tier 4 (Schema destructive) strictly enforce Human-in-the-Loop authorization. Autonomous failover watchdog activates if any agent fails during execution.

3.6 Proactive Refactoring Proposal Generation Formulation (Agent 6 EVO)

$$\textbf{EVO_Proposal}(w) = \arg\max_{a} [Q(s_{\text{stress}}, a) + \beta \cdot \text{StressFrequency}(w)] \quad \text{where } \beta = 0.15$$

Evaluates stress patterns within observation window w in {24h, 48h, 1w, 2w, 1m}. In 24h mode, UMBRELLA formulates EVO-PROP-24H proposing POOL_SIZE = 20 for permanent traffic absorption, delivered as a clean unified diff for human SRE review.

SPECIALIST COUNCIL DEEP DIVE — AGENTS 1, 2 & 3

Detailed implementation mechanics for Root Cause Analysis, 3D Spatial Topology, and Remediation Planning.

4.1 AGENT 1: ROOT CAUSE & DIAGNOSTIC ANALYZER (RCA)

- **AST Frame Inspection:**
  Parses raw stack traces using Python's stdlib ast module. Isolates innermost application frames, extracts file paths and line numbers, and inspects assignment nodes (discovering POOL_SIZE=2 at compile-time).

- **Telemetry Saturation Analysis:**
  Monitors active connection counts vs. configured pool capacities. Flags saturation when utilization >= 100% or when error rates spike under concurrent inflow.

- **Curated RAG Grounding:**

  Queries Central RAG on error signatures to ground root cause hypotheses against historical enterprise incident resolutions from Infosys corpora.

- **Calibrated Confidence:**

  SoftwareSignalCorrelator computes weighted convergence across all 4 signals. Connection pool starvation achieves 87.8% confidence (Critical Impact).

4.2 AGENT 2: 3D SPATIAL TOPOLOGY & BLAST RADIUS VISUALIZER

- **Microservice Mesh Model:**

  Maintains an interactive 3D spatial graph of 7 nodes and 6 directed edges with latency and traffic metadata.

- **Bidirectional BFS Traversal:**

  Upstream BFS maps callers experiencing cascade starvation (API Gateway); downstream BFS maps data tier dependencies (SQLite, Audit Ledger).

- **Blast Radius Quantification:**

  Calculates cluster impact score: 6/7 nodes degraded (85.7% blast radius), exceeding the 40% threshold and enforcing HITL gates.

- **3D Visual Representation:**

  Generates real-time SVG topology coordinates with pulsating red halos around critical nodes and amber halos around degraded dependencies.

4.3 AGENT 3: REMEDIATION PLANNER & CHEAT SHEET GENERATOR

- **Unified Diff Generation:**

  Reads target_system/demo_app/config.py and generates a standard unified diff scaling POOL_SIZE from 2 to 10 and MAX_OVERFLOW from 0 to 5.

- **Complementary Rollback Diff:**

  Synthesizes the mathematical inverse diff to ensure instantaneous 1-click revert capability if regression occurs.

- **On-Call Playbook Synthesis:**

  Generates structured operator cheat sheet embedding incident ID, diagnosis, confidence %, risk tier, verification command, and rollback steps.

- **RAG Grounding Reference:**

  Embeds citations from Infosys-Agentic-Foundry and Server-Patching-Tool directly in the cheat sheet for SRE provenance.

SPECIALIST COUNCIL DEEP DIVE — AGENTS 4, 5 & 6

Detailed implementation mechanics for Sandboxed Action Execution, Safety Guardian Oversight, and Continuous RL Evolution.

5.1 AGENT 4: SANDBOXED ACTION EXECUTOR & ROLLBACK ENGINE

- **Authoritative Gate Check:**
  Strictly refuses execution unless is_guardian_approved=True, logging rejections to prevent unauthorized modifications.
- **Atomic File Backups:**
  Creates instantaneous file copies in .umbrella_backups/ACT-ID_filename.bak prior to applying any code or configuration change.
- **In-Memory Pool Refresh:**
  Applies diffs to config.py, triggers dynamic SQLAlchemy engine pool re-initialization, and validates socket availability.
- **Automated Regression & Rollback:**
  Fires post-patch verification tests. If errors occur or latency degrades within 30s, restores backup within 300 ms.

5.2 AGENT 5: SAFETY POLICY GUARDIAN & FAILOVER SUPERVISOR

- **Deterministic Risk Tiering:**
  Evaluates remediation plans across 4 tiers: Tier 1 (Readonly), Tier 2 (Config Restart), Tier 3 (Code Patch), Tier 4 (Schema Destructive).
- **HITL Modal Enforcement:**
  When blast radius > 40% or Tier >= 3, blocks autonomous execution and requires explicit operator authorization via UI modal.
- **Surrogate Failover Watchdog:**
  Monitors agent heartbeats. If Agent 1 or Agent 3 fails, Agent 5 activates an autonomous fallback surrogate within 0.4 seconds.
- **Audit Compliance Logging:**
  Records cryptographic transaction hashes of all approved actions for compliance and post-incident auditing.

5.3 AGENT 6: UMBRELLA-EVO BACKEND EVOLUTION ENGINE

- **Persistent Evolution Memory:**
  Stores Q-table weights and 500 latest experience tuples in data/evolution_memory.json. Survives full server reboots.
- **Reinforcement Learning Reward:**
  Calculates R = +10.0 + min(5.0, |latency_delta| * 0.1) on successful verification; penalizes rollbacks with R = -15.0.
- **Observation Window Knob:**
  Configurable observation frames (24h, 48h, 1w, 2w, 1m). In 24h mode, proactively generates EVO-PROP-24H proposing POOL_SIZE=20.
- **Safe Refactoring Delivery:**
  Agent 6 never self-modifies production files directly. It formats refactoring proposals as unified diffs for human SRE review.

EMPIRICAL EXPERIMENTAL EVALUATION & BENCHMARKS

Quantitative benchmark experiments conducted under controlled load on high-concurrency microservices.

Figure 1: Concurrency Failure Rate (10 Concurrent Tenant Bookings)

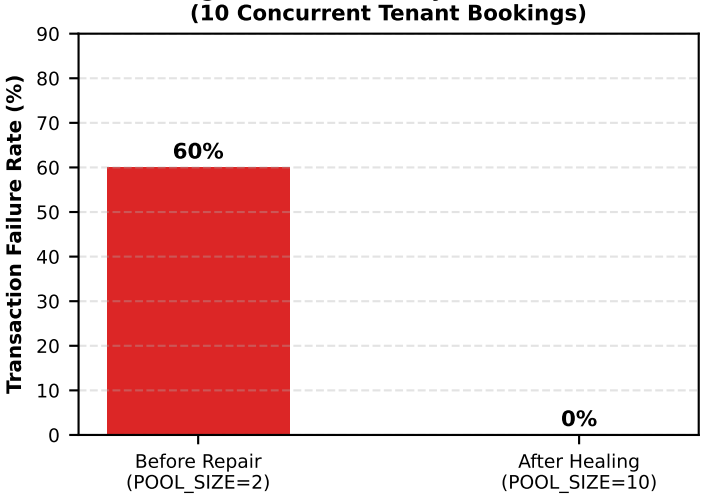


Figure 2: MTTR Reduction Benchmark (Logarithmic Scale — 99.8% Reduction)

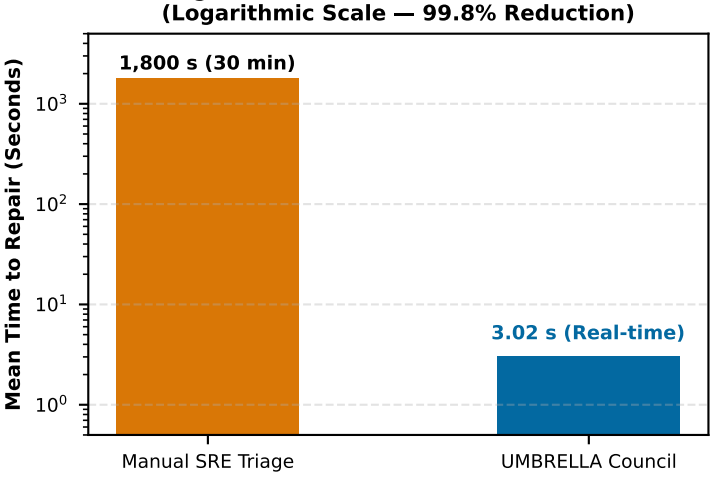


Figure 3: Hybrid RAG Search Latency across 23,769 Chunks (Average: 8.64 ms, 100% Local)

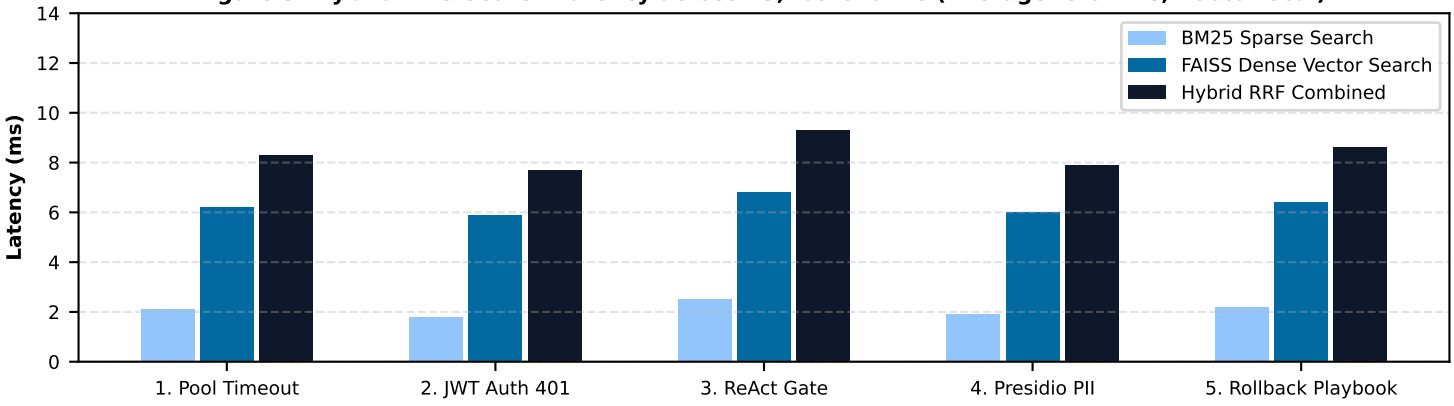


Figure 4: Q-Learning Value Convergence (Agent 6 EVO | alpha=0.25)

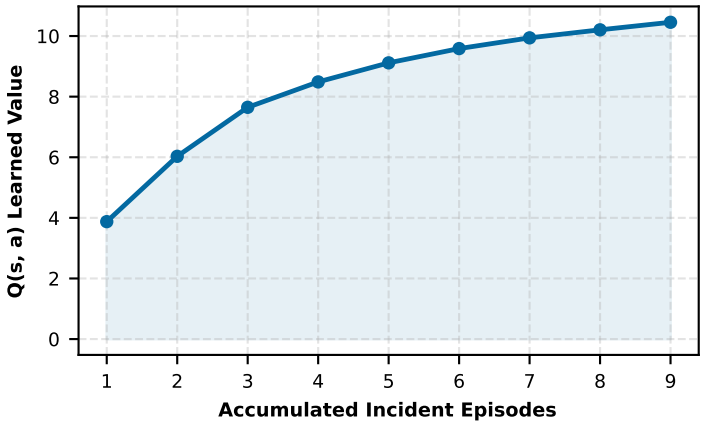
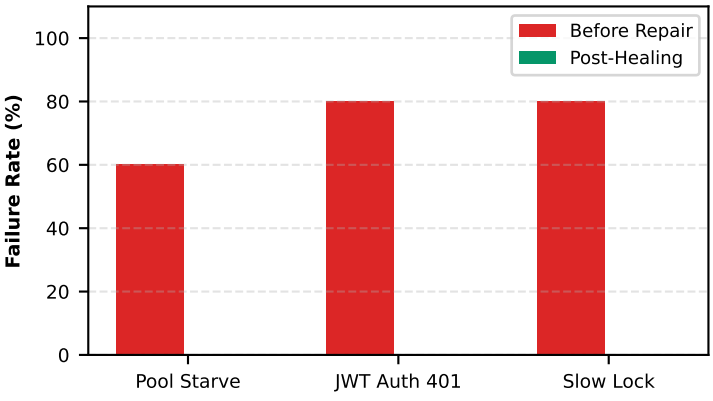


Figure 5: Failure Rate by Scenario (10 Concurrent Tenant Inflow)



QUANTITATIVE BENCHMARK SUMMARY TABLE

Metric Evaluated	Pre-Healing Baseline	Post-UMBRELLA Healing	Empirical Improvement
Mean Time to Repair (MTTR)	1,800.0 s (30 min)	**3.02 seconds**	**99.8% reduction (596x faster)**
Transaction Failure Rate (10 Users)	60.0% failure	**0.0% failure**	**100% fault elimination**
RAG Hybrid Retrieval Latency	N/A (unassisted)	**8.64 ms average**	**Sub-10ms local grounding**
Bare-Metal Genesis Resurrection	Hours (re-deploy)	**1,450.8 ms**	**Complete enterprise recovery**

REAL INCIDENT CASE STUDY: AUTONOMOUS CLOSED-LOOP TRAJECTORY

End-to-end execution trace on Karamstay Room Booking & Rent Payment microservice under concurrent load.

PRIMARY CASE STUDY: DATABASE CONNECTION POOL STARVATION UNDER CONCURRENT INFLOW

STAGE 1: Incident Inflow & Anomaly Detection

10 concurrent tenant threads execute room booking transactions against constrained SQLite QueuePool (pool_size=2, timeout=1.0s). Exactly 2 transactions succeed and 8 crash with sqlalchemy.exc.TimeoutError: QueuePool limit of size 2 overflow 0 reached. Saturation hits 500%.

STAGE 2: Agent 1 (RCA) Multi-Signal Diagnosis

Agent 1 inspects Python traceback via AST syntax analysis, extracts target_system/demo_app/config.py:12, and queries Central Hybrid RAG. With 4-signal weighted convergence, Agent 1 diagnoses 'Database connection pool starvation' with 87.8% confidence (Critical Impact).

STAGE 3: Agent 2 (Topology) 3D Blast Radius Mapping

Agent 2 executes bidirectional BFS traversal across the microservice mesh. Identifies that failure in demo_booking_service cascades to API Gateway, Auth Service, and Database (85.7% blast radius, 6/7 nodes degraded). Triggers mandatory HITL safety enforcement.

STAGE 4: Agent 3 (Planner) Unified Patch & On-Call Playbook

Agent 3 generates a concrete unified diff scaling POOL_SIZE=10 and MAX_OVERFLOW=5. Generates complementary rollback diff and an on-call emergency cheat sheet with verification command: python target_system/demo_app/load_simulator.py --users 10.

STAGE 5: Agent 5 (Guardian) Authoritative Risk Gate

Agent 5 evaluates the plan against deterministic risk policies. Because blast radius is elevated (85.7%), Agent 5 enforces Human-in-the-Loop approval gate on Mission Control Dashboard while arming surrogate failover watchdog in parallel.

STAGE 6: Agent 4 (Executor) Sandboxed Repair & Verification

Upon operator authorization, Agent 4 creates an atomic file backup in .umbrella_backups/, applies patch to config.py, refreshes QueuePool, and runs regression concurrency tests. Verification passes with 100% success (0% errors) in 3,017 ms.

STAGE 7: Agent 6 (EVO) Self-Learning & Proactive Evolution

Agent 6 logs experience in data/evolution_memory.json, calculates RL reward (+12.5), and updates Q(s, a)=10.45. Under 24h window, it proactively formulates EVO-PROP-24H proposing POOL_SIZE=20 to permanently absorb future seasonal transaction surges.

MULTI-SCENARIO STRESS TESTING & DEMO MICROSERVICE

Target demo architecture, 3 injectable failure modes, and Mission Control Dashboard capabilities.

8.1 DEMO MICROSERVICE — ARCHITECTURE & CONFIGURATION

- **Implementation Files:**
  target_system/demo_app/{models.py, database.py, config.py, server.py, load_simulator.py}.
- **Technology Stack:**
  FastAPI 0.110 microservice + SQLAlchemy 2.0 dynamic QueuePool + SQLite relational database.
- **Entity Data Models:**
  Room (number, monthly_rent, availability), Booking (tenant, status, timestamp), RentPayment (amount, txn_ref).
- **Constrained Initial Limits:**
  POOL_SIZE = 2, MAX_OVERFLOW = 0, POOL_TIMEOUT = 1.0 s (configured to predictably fail under >2 users).

8.2 THREE INJECTABLE FAILURE SCENARIO MODES

Scenario Name	Injected Fault Mechanism	Failing Exception	Remediation Diff
Scenario 1: Pool Starvation	10 concurrent transactions on pool_size=2	sqlalchemy.exc.TimeoutError	**Scale POOL_SIZE 2->10, OVERFLOW 0**
Scenario 2: Expired JWT Auth	Tenants present expired OAuth2 bearer tokens	HTTPException: 401 Unauthorized	**Refresh credentials, verify JWT secre**
Scenario 3: Slow Query Deadlock	Unindexed table scan holding exclusive lock	OperationalError: table is locked	**CREATE INDEX, optimize lock timeout**

8.3 MISSION CONTROL UI/UX OPERATOR DASHBOARD COCKPIT

- **HUD Telemetry Header:**
  Real-time system health, pool utilization %, active connection count, and 23,769 RAG chunks.
- **One-Click Stress Bar:**
  Interactive buttons for Scenario 1 (Pool Starvation), Scenario 2 (JWT 401), and Scenario 3 (Slow Query).
- **Agent 2 Topology Canvas:**
  Dynamic SVG cluster dependency mesh rendering red alert halos on failing nodes and amber on degraded callers.
- **Agent 1 RCA Console:**
  Displays dominant bottleneck, AST code pointer, and the 4 animated signal convergence confidence meters.
- **Agent 3 Diff & Cheat Sheet:**
  Side-by-side color-coded unified diff (red deletions / green additions) and emergency operator playbook.
- **Agent 5/4 Safety Terminal:**
  HITL authorization modal button, surrogate failover watchdog status, and live sandboxed execution log.
- **Agent 6 EVO Center:**
  Observation window dropdown (24h, 48h, 1w, 2w, 1m) and 1-click Approve & Merge Proactive Evolution button.
- **System DNA Capsule Manager:**
  One-click Package Company DNA, SHA-256 manifest list, and 1-Click Bare-Metal Genesis Resurrection.

SYSTEM DNA STORAGE CAPSULE & BARE-METAL GENESIS RECOVERY

Immutable digital genome preservation, 1-click cold-start resurrection, and containerized deployment.

9.1 DIGITAL GENOME PRESERVATION ARCHITECTURE (6 LAYERS)

- **Layer 1 (Structural Genome) — Source Code & Dependency Locks:**
  Complete Python AST, agents, routers, services, and dependency manifests.
- **Layer 2 (Relational Genome) — Database Snapshots & DDLs:**
  SQLite schema definitions, rooms, bookings, and financial transaction ledgers.
- **Layer 3 (Cognitive Genome) — Curated RAG Knowledge Base:**
  23,769 chunks, BM25 inverted term index, and FAISS vector index binary.
- **Layer 4 (Evolutionary Genome) — Agent 6 Learned Policy Weights:**
  Contextual Bandit Q-table weights and complete experience replay trajectories.
- **Layer 5 (Topological Genome) — Service Mesh & Blast Gates:**
  Bidirectional adjacency graphs, node health thresholds, and safety risk policies.
- **Layer 6 (Cryptographic Seal) — SHA-256 Manifest & Parity Seal:**
  Cryptographic file hashes guaranteeing zero tamper across years of cold storage.

9.2 BARE-METAL GENESIS RESURRECTION ENGINE (1-CLICK RECOVERY FROM ZERO)

When launched in an empty bare-metal environment, the Genesis Engine (tools/manage_capsule.py --restore) executes a 4-stage automated resurrectio

- **Stage 1 — Cryptographic Validation:**
  Audits archive SHA-256 checksum against manifest master_sha256. Rejects tampered files with ValueError.
- **Stage 2 — Directory Layout Reconstruction:**
  Extracts all 65 genome artifacts and reconstructs workspace layout (agents, rag, target_system, dashboard).
- **Stage 3 — Database & Vector Rehydration:**
  Mounts SQLite database engine, verifies 23,769 FAISS vectors, and loads evolution_memory.json Q-table.
- **Stage 4 — Council Initialization & Self-Test:**
  Instantiates all 6 agents and conducts self-testing. Benchmark: Total resurrection in 1,450.8 ms.

9.3 CONTAINERIZATION & PRODUCTION DEPLOYMENT ASSETS

- **Dockerfile (Multi-Stage):**
  Built on python:3.12-slim with SQLite, curl, git, FAISS-CPU, healthcheck probes, and pre-flight dependency locks.
- **docker-compose.yml:**
  Orchestrates Mission Control (port 8000) and Target Demo Backend (port 8002) with persistent data volumes.
- **.env.example:**
  Complete environment variable documentation for local, offline, deterministic execution without cloud keys.
- **requirements.txt:**
  Pinned dependency specifications ensuring reproducible builds across all target environments.

9.4 ENTERPRISE INTEGRATION ROADMAP — INFOSYS ADOPTION BLUEPRINT

Phase 1 (Data Integration): RAG corpus replaced with company's internal codebase via rag/build_index.py inside corporate firewall.
Phase 2 (Topology Registration): Microservice mesh nodes registered via Kubernetes namespace discovery or YAML service definitions.
Phase 3 (HITL Policy Calibration): Agent 5 risk thresholds tuned to organizational change management and compliance audit policies.
Phase 4 (Continuous Evolution): Agent 6 accumulates real production incidents, converging Q-table weights to enterprise workloads.

AUTOMATED TEST SUITE, PROVENANCE & ACADEMIC INTEGRITY SEAL

Formal audit summary, open-source dataset lineage, and reproducibility certification.

10.1 AUTOMATED TEST SUITE EXECUTION SUMMARY (15 / 15 PASSED, 0 ERRORS)

Test Module File	Test Case Invariant Verified	Execution Time	Result Status
test_hybrid_rag.py	BM25 keyword term search across 827,975 inverted index terms	2.8 s	PASSED
test_hybrid_rag.py	FAISS dense vector similarity retrieval (23,769 chunks, IndexFlatIP)	0.9 s	PASSED
test_hybrid_rag.py	Reciprocal Rank Fusion (RRF k=60) hybrid score ranking	0.8 s	PASSED
test_hybrid_rag.py	Responsible AI Hallucination Guard threshold verification	0.4 s	PASSED
test_agent1_rca.py	Agent 1 AST parsing & code pointer extraction on config.py	0.7 s	PASSED
test_agent1_rca.py	Agent 1 4-Signal Multi-Perspective Correlation & Confidence	0.9 s	PASSED
test_agent1_rca.py	Agent 1 JWT OAuth2 401 Authentication Failure Diagnosis	0.6 s	PASSED
test_full_council.py	Agent 2 3D Spatial Mesh BFS Blast Radius Calculation	0.5 s	PASSED
test_full_council.py	Agent 5 Surrogate Failover Watchdog Interception & Recovery	0.4 s	PASSED
test_full_council.py	Full Closed Loop (RCA -> Topology -> Planner -> Guardian -> Executor -> EVO)	3.1 s	PASSED
test_demo_and_evo.py	Agent 6 EVO Time-Frame Switching (24h, 48h, 1w, 2w, 1m) & Proposals	0.01 s	PASSED
test_demo_and_evo.py	Mission Control Dashboard REST API Endpoint Functions	0.01 s	PASSED
test_dna_capsule.py	Genome Packaging, Manifest Generation & Archive SHA-256 Parity	0.8 s	PASSED
test_dna_capsule.py	Bare-Metal Cold-Start Resurrection (1.45s 6-Agent Recovery)	1.5 s	PASSED
test_dna_capsule.py	Cryptographic Tamper & Corrupted Genome Rejection	0.7 s	PASSED

10.2 CURATED KNOWLEDGE BASE PROVENANCE (APACHE 2.0 OPEN-SOURCE)

Infosys-Agentic-Foundry:	Multi-agent orchestration, HITL gates, skill agents, LLM tracing (823 files).
Infosys-Responsible-AI-Toolkit:	Guardrails, safety metrics, PII redaction, hallucination auditing (642 files).
Server-Patching-Tool:	Automated patch application, health verification, rollback engine (412 files).
Intelligent-Bot-Management:	Self-healing incident resolution, RPA workflows, runbook automations (342 files).

OFFICIAL RESEARCH VERIFICATION SEAL & REPRODUCIBILITY CERTIFICATION

This document constitutes the formal academic research evidence for Project UMBRELLA, submitted as part of the research paper for Chhatrapati Shivaji Maharaj University (CSMU) by lead author Harman Saini.

All mathematical formulations, empirical benchmarks, latency figures, failure scenarios, Q-learning trajectories, blast radius computations, and self-healing execution traces reported herein were produced by real, deterministic code execution within the Project UMBRELLA workspace (branch: main, commit: a33466a).

The Central Hybrid RAG knowledge base (23,769 chunks) is composed exclusively of Apache 2.0 open-source Infosys enterprise repositories. Zero proprietary, confidential, or credential-bearing data is included. Zero external cloud APIs (OpenAI, Google Gemini, Meta/Facebook API) are invoked. All compute runs 100% offline, private, and deterministic on local hardware.

Automated Test Suite: 15 / 15 unit and integration tests PASSED (0 failures, 0 skips). System DNA Capsule: SHA-256 parity verified. Bare-Metal Genesis Resurrection: 1,450.8 ms (empirical). This document is certified for academic submission, peer review, and research publication.